LiteTest Runner Internal Reference

Document Version: 1.0

Runner Version: 2026-06-11

Test Version: 1.0.0

This document provides a reference to the LiteTest Runner API internals. It includes types, structs, unions, enums, macros, and functions.

Preface

For a list of other LiteTest documents and the LiteTest repository layout, see the LiteTest Documentation Guide.

For a glossary of terms, see the LiteTest Glossary Reference.

Document Version History

Document	Date	LiteTest	Description
1.0	2026-06-11	1.0.0	Initial LiteTest Framework Reference.

The **Document** column tracks the version `M.u` (major, update) of this Framework Reference document. The **LiteTest** column records the latest LiteTest version at the time this document version was published.

The current LiteTest version is defined in `litetest_runner.h` by the macro `LT_VERSION`, which specifies a string of the form `"M.m.p"` (major, minor, patch). `litetest_runner.c` defines a matching version string `LT_VERSION_C`. For details, see the public API and framework documentation.

A **major LiteTest release requires a corresponding major update to this document** and therefore the document's major version must match the LiteTest major version. The **update** version tracks updates to this document itself and does not correspond to LiteTest minor or patch versions. The update version is incremented whenever this document is updated without a change to the major version, and it resets to `0` when the major version increases.

Documentation Style Guide

1. Tone
   - Technical, precise, and neutral.
   - No marketing language.
   - Prefer clarity over cleverness.
2. Formatting
   - Use backticks for code identifiers.
   - Use fenced code blocks for file trees, examples, and commands.
   - Keep line lengths reasonable for GitHub rendering.
3. Writing Guidelines
   - Define terms once, and then use them consistently.
   - Avoid synonyms for technical concepts (e.g., always "update version," never "revision").
   - Keep paragraphs short.
   - Use lists for enumerations.
4. `Click to view`

- Use `Click to view` sections and subsections to keep the document readable while still accommodating large amounts of technical detail:
- Collapsing sections allows readers to scan the structure and expand only what they need.
- This keeps the document manageable, avoids overwhelming readers with unrelated detail, and makes the document easier to navigate.

5. Writing Style Consistency: To keep LiteTest documentation clear and easy to read, maintain **parallel structure** within lists and related sentences. In practice:
   - Start list items with the same part of speech (typically a verb).
   - Keep grammatical patterns consistent across bullets.
   - Avoid mixing styles such as "Keep paragraphs short" with "Using lists for enumerations."
   - Rewrite items as needed so the list reads smoothly and uniformly.

Table of Contents

1. Overview

LiteTest's internal architecture consists of several cooperating subsystems, including the implementation of API macros and functions, the execution engine, guard/fault handling, isolation support, file/path utilities, matching/comparison helpers, and environment support. These components work together to run test groups and test expressions reliably across threads and processes while capturing faults and producing structured reports.

The remainder of this document describes each internal symbol used to implement these subsystems, organized from high-level behavior down to low-level details.

In this document, a *symbol* refers to any named entity in the LiteTest framework, including:

- typedefs
- structs
- enums and enum values
- macros
- variables
- functions

This document is organized by the named entities in the LiteTest Runner implementation. Each symbol (type, macro, function, etc.) is described individually, including its purpose, behavior, and usage. It serves as the reference companion to the LiteTest Framework Guide, defining the framework's components precisely while the Guide explains their design and interaction.

Public and Internal Naming Conventions Any framework names that are public and visible to LiteTest API users are prefixed with `lt_...` (typically lowercase) or `LT_...` (typically uppercase).

Any framework names that are internal (but technically visible to LiteTest API users) follow the pattern `litetest_..._internal_t`, `litetest_..._internal`, or `LITETEST_..._INTERNAL`. These names should not be referenced by API users.

In general, users of the API should not define names prefixed with `lt_`, `LT`, `litetest_`, or `LITETEST`, or reference names prefixed with `litetest_` or `LITETEST_`.

2. Symbols Defined in `litetest_runner.h`

These are used by `litetest_runner.h` and `litetest_runner.c`, and are public or internal based on their name per the naming conventions.

2.1. Types

These typedefs declare fundamental internal types used throughout the LiteTest framework.

lt_result_t Declaration

```
typedef struct
{ size_t pass;
  size_t fail;
  size_t fault;
  size_t injected_fail;
  size_t injected_fault;
} lt_result_t;
```

Description
Result counters produced by test execution and propagated through group and orchestrator aggregation logic.

Usage Notes
- `fault == SIZE_MAX` is a sentinel meaning a function-level fault was captured. - Function-level fault sentinel values are introduced by the signal-guard path. - This sentinel is distinct from ordinary fault counts returned by test logic. - Related helpers include `lt_currentresult(void)` and result-merging macros.

Example

```
lt_result_t result = {0, 0, 0, 0, 0};
```

2.2. Structs

struct Declaration

```
typedef struct <StructName> {
    <field-type> <field-name>;
    ...
} <StructName>;
```

Description
Explain the purpose of this struct, what data it aggregates, and how it participates in the LiteTest execution model.

Fields - `<field-name>` — description
- `<field-name>` — description

Usage Notes
- Ownership or lifetime rules
- Whether fields must be initialized by the user or framework

Example

```
<StructName> s = { ... };
```

2.3. Enums and Enum Values

enum Declaration

```
typedef enum <EnumName> {
    <ENUM_VALUE_1>,
    <ENUM_VALUE_2>,
    ...
} <EnumName>;
```

Description

Explain what conceptual category this enum models and how the values are used by the framework.

Values - `<ENUM_VALUE_1>` — meaning
- `<ENUM_VALUE_2>` — meaning

Usage Notes

- Any ordering assumptions
- Whether values map to external formats (strings, logs, etc.)

Example

```
<EnumName> mode = <ENUM_VALUE_1>;
```

2.4. Global Variables

Declaration

```
extern <type> <VariableName>;
```

Description

Explain the purpose of this global variable, what state it represents, and how it is used by the LiteTest framework.

Usage Notes

- Whether the variable is read-only or writable
- Whether users are expected to modify it
- Thread-safety considerations
- Lifetime and initialization rules

Example

```
if (<VariableName> == ...) {
    ...
}
```

2.5. Macros

Definition

```
#define <MACRO_NAME>(...) <expansion>
```

Description
Describe the purpose of this macro, what it expands to conceptually, and how it fits into the LiteTest orchestration or test definition model.

Parameters - `<param>` — meaning and constraints
- `<param>` — meaning

Usage Notes
- Side effects
- Evaluation rules (e.g., multiple evaluation hazards)
- Whether arguments must be constant expressions

Example

```
<MACRO_NAME>(arg1, arg2);
```

2.6. Functions

The functions in this section may be `static`, `static inline`, or (by default) `extern`, depending on how they are used within the framework.

() Signature

```
<return-type> <FunctionName>(<parameters>);
```

Description
Explain what this function does, when it is called, and how it interacts with the LiteTest runtime.

Parameters - `<param-name>` — meaning, constraints, ownership
- `<param-name>` — meaning

Return Value
Describe what is returned and under what conditions.

Errors / Preconditions
- Preconditions the caller must satisfy
- Error conditions or undefined behavior cases

Usage Notes
- Thread safety
- Lifetime rules
- Interaction with other LiteTest components

Example

```
<return-type> result = <FunctionName>(...);
```

3. Symbols Defined in `litetest_runner.c`

These symbols are local to `litetest_runner.c` unless specified or defaulting to extern. Symbols that are local do not have to conform to the internal naming conventions and more natural names may be used.

3.1. Types

(Placeholder)

3.2. Structs

(Placeholder)

3.3. Enums and Enum Values

(Placeholder)

3.4. Global Variables

(Placeholder)

3.5. Macros

(Placeholder)

3.6. Functions (declared as a forward reference) in `litetest_runner.h`

These functions are referenced internally by `litetest_runner.h` but defined in `litetest_runner.c`. These functions are (by default) extern and must conform to the internal name conventions.

(Placeholder)

3.7. Functions (not declared in `litetest_runner.h`)

These functions are local to `litetest_runner.c` and defined as static. Since these are local, these names do not have to conform to the internal name conventions and more natural names may be used.

(Placeholder)

4. Execution Engine

(Placeholder for execution helpers.)

5. Signal Handling

(Placeholder for signal guard helpers.)

6. Process Management

(Placeholder for fork/exec/wait logic.)

7. Thread Management

(Placeholder for pthread logic.)

8. File and Path Support

(Placeholder for filesystem support helpers.)

9. Matching and Comparison Support

(Placeholder for comparison support helpers.)

10. Environment Support

(Placeholder for environment support helpers.)

Repository Layout

GitHub repository: `paulsinclair51/LiteTest`

Repository layout (listing core files):

```
LiteTest/
|- .github/
|  \- workflows/
|     \- ci.yml
|- README.md
|- LICENSE
|- Makefile
|- build_test_litetest.ps1
|- build/
|- docs/
|  \- LiteTest_API_User_Guide.md
|  \- LiteTest_API_Reference.md
|  \- LiteTest_Contributor_Guide.md
|  \- LiteTest_Framework_Guide.md
|  \- LiteTest_Framework_Reference.md
|- examples/
|- include/
|  \- litetest_runner.h
|- reports/
|  |- litetest_test_report-I.txt
|  \- litetest_test_report.txt
|- scripts/
|- src/
|  \- litetest_runner.c
|- tests/
|  |- test_litetest.c
|  |- test_orchestrator.c
|  |- test_guard1.c
|  \- test_guard2.c
```

Use this as a reference when adapting LiteTest into your own project structure.

Glossary

General Terms

- `API`: Application Programming Interface.
- `category`: A labeled set of `LT_GROUP` and `LT_TEST` macros whose combined results are written by `LT_WRITE_RESULT` to the report with a specified category name.
- `control file`: A previously generated file that can be compared to a newly generated file for differences. Differences (other than expected ones like timestamps) typically indicate a test failure. Sometimes the control file is out of date and must be replaced by promoting the new file.
- `customization support functions`: API functions provided to support customizing the orchestrator and test group functions.
- `default report filename`: The report filename LiteTest uses when only a directory path (or no `PATH`) is provided.
- `executable`: The compiled test program that runs the orchestrator and test functions.
- `fail`: A counted test failure where the `LT_TEST` or `LT_INJECT_TEST` expression evaluates to zero.
- `fault`: A counted runtime fault captured by LiteTest guards (e.g., invalid memory access).
- `Concurrent block`: A set of tests bracketed by `LT_BEGIN_CONCURRENT` and `LT_END_CONCURRENT`.
- `group`: See `test group`.
- `guard`: The protection mechanism used to catch runtime faults and continue test execution.
- `guard level`: The nesting depth of active guards while test groups and tests run.
- `inject mode (-i)`: Optional command-line flag that enables an `LT_INJECT_TEST` to be executed.
- `isolation`: Execution mode for `LT_GROUP`, `LT_TEST`, or `LT_INJECT_TEST`. `0` = same thread, `1` = separate thread, `2` = separate process.
- `maxargs`: Maximum number of command-line arguments accepted by orchestrator parsing. `LT_PARSE_ARGS` handles the first two arguments; additional arguments must be parsed by custom code.
- `maxparallel`: Upper bound on concurrent `LT_GROUP`, `LT_TEST`, `LT_INJECT_TEST`. Set in `LT_INIT_ORCHESTRATOR` and `LT_GROUP`.
- `notes`: Optional text appended to the report by `LT_CLOSE_REPORT`.
- `orchestrator`: The `main` function that initializes LiteTest, runs groups or tests, and writes report output.
- `pass`: A counted successful test where the expression evaluates to non-zero.
- `PATH`: Optional command-line output destination; may be a report file path or directory path.
- `process isolation`: Isolation mode where a test group or test runs in a separate process.
- `project`: Project identifier used in orchestrator initialization and default report naming.
- `test group`: A grouping of `LT_TEST` and optionally nested `LT_GROUP` macros.
- `test group function`: A function declared with `LT_DECLARE_GROUP` that contains `LT_TEST` and `LT_GROUP` macros.

- `thread isolation`: Isolation mode where a test/assert call runs in a separate thread.
- `test case`: Not used in LiteTest. In other contexts it may mean a single test or a set of tests; LiteTest uses "test expression" and "test group."
- `test`: See `test expression`.
- `test expression`: An expression passed to an `LT_TEST` macro that can be cast to `int`; zero means fail, non-zero means pass.
- `testing artifact`: A file or output generated by the test executable (e.g., test report, stdout, stderr).
- `testing function`: A user-written function used to implement or support a test expression.
- `test support functions`: API functions provided to simplify writing tests.
- `title`: Optional report header text provided when opening the report.
- `Public API`: Functions, macros, and types intended for external use.
- `Internal API`: Framework-only symbols not intended for external use.
- `Semantic Versioning`: Versioning scheme using `M.m.p`.
- `Report Format`: The output structure produced by LiteTest test runs.

Framework-Specific Terms

- **Orchestrator Lifecycle**: The sequence of initialization, group execution, test execution, and report finalization performed by the LiteTest framework.
- **Guard Behavior**: The mechanism LiteTest uses to catch runtime faults (e.g., segmentation faults) and continue executing remaining tests.
- **Isolation Semantics**: The rules governing how tests and groups run in threads or processes to prevent interference and ensure fault containment.
- **Concurrency Model**: The framework's rules for running tests concurrently within `LT_BEGIN_CONCURRENT` / `LT_END_CONCURRENT` blocks.
- **Execution Phases**: The internal stages of orchestrator operation, including initialization, argument parsing, group dispatch, test dispatch, and report writing.
- **Fault Handling Model**: The framework's strategy for capturing and reporting faults without terminating the entire test run.
- **Control File Promotion**: The process of replacing an outdated control file with a newly generated file when differences are expected or intentional.
- **Nested Group Behavior**: The rules governing how groups may contain other groups and how isolation and concurrency propagate through nested structures.
- **Concurrent Block Behavior**: The semantics of executing multiple tests in parallel within a concurrent block, including ordering and isolation rules.